DD249U - Report

Jack Lindberg, Filip Redhammar, Carl Tydén

October 2026

1 Problem Description

The control flow or execution path of a program determines in which order the code of the program executes. Common techniques of altering the control flow of a program includes function calls and jump statements. Function calls transfers the execution of the program to a packaged unit of code with a defined interface called a function. After the function has finished, execution is transferred back to where the function call was made. Jump statements similarly to function calls allows for transfers of execution to other parts of the memory, however unlike function calls jump statements make no guarantee of a return of the execution. Since function calling involves two transfers of execution it is useful to have terminology distinguishing the two types. The first transfer, transferring the execution into the function is commonly referred to as *forward edge control flow* whilst the transfer back is referred to as *backward edge control flow*.

When an attacker is able to overwrite parts of the memory of a program (by using a buffer overflow for an example), they may be able to hijack the control flow. This could be used by the attacker to run their own injected code (*shellcode*) or to run code in other parts of the program or existing libraries. This can often allow an attacker to perform arbitrary actions on a target without injecting any of their own shellcode. When this is achieved by overwriting the return address it is referred to as *Return Oriented Programming (ROP)*. If instead the address of the function call is overwritten to achieve a similar result it is referred to as *Call Oriented Programming (COP)*.

2 Technology

There are several methods available in order to mitigate the problems described above. One fairly common method is to implement control flow integrity checks(CFI) during runtime. These checks are used to validate the addresses when either entering a function or exiting a function. As the transfer works differently so does these checks.

2.1 Forward edge CFI

Forward edge control flow integrity is a method used for verifying that the addresses that the program jumps to are in accordance with how the program is intended to run. Many variants exists that mitigate different control flow altering forms. In this project pointer authentication is used. Pointer authentication is used to verify integrity of function pointers. This is achieved using cryptographic signatures of the pointers that are used to prove that a pointer is genuine.

2.2 Backward edge CFI

As for backward edge implementation goes, this also works using the return address for verification. This works by capturing the address of the called function right after entering the function, as well as the return address just before returning from the same function. These addresses are captured using

the macro, implemented in the source code of the program at compile time. The underlying verifier saves the address during the duration of the function execution and compares the newly acquired address before return with the previously saved address. The underlying verifier/monitor takes action against the program if it has been tampered with. Alternatively the addresses are unchanged and the verifier/monitor allows the program to proceed without intervention.

3 Implementation Details

3.1 Backward Edge

The shadow stack is implemented with a standard stack data structure that is based on fixed size arrays. The stack is used differently depending on if its used in the kernel or in a monitor. The difference comes down to how and where the stacks are initialized.

To make sure the stack is updated there has to be some boilerplate call in each function. This boilerplate must include a call in the beginning of a function that `pushes`, to the shadow stack, the correct return address before any other operation is made, ensuring a known good return address. The other part of the boilerplate has to be inserted **just before** the functions return statement. This is where a comparison is made with the known good return address that was saved at the beginning of execution of the function. In other words this part has to pop from the shadow stack and compare it.

Both of these parts have the shared requirement of having to be run at very specific times in a functions execution and without ever letting a malicious actor have any action that in any way alters the behavior. As it would otherwise be possible to change the address after the comparison and simply circumvent the control altogether.

The solution to this problem was using 2 macros that are run in each function execution. `CFI_ENTER` and `CFI_RETURN`. These macros are run first and last respectively in each function.

- `CFI_ENTER` pushes the known good return address to the shadow stack, grabbed from the return address register.
- `CFI_RETURN` pops the top value of the shadow stack, comparing it to the current one and based on the result of that comparison either; returns or does some other action.

These two macros can then be customized to take any action on both steps. Whether running a system or running any other function such as stopping the process on a mismatch.

3.1.1 Monitor

One of the implementations includes a standalone monitor that works as a separate application running above the kernel. This makes monitoring easy and reliable without having to alter with the kernel structure. It does take more memory space and potentially runs slower compared to the kernel implementation above. Below are some crutial details related to the monitor implementation.

`IPC:` Inter process communication functionality is used by applications to communicate with each other. This method is a native functionality in s3k. A socket is setup, which gives us the capability to get information from the user application to the monitor app. In order for the monitor to make calculations on the values provided it is neccessary for us to utilize the following structure below.

Monitor:

```
receive = s3k_sock_recv(socket, free_cap_idx);
... comparison
s3k_sock_send(socket, &reply);
```

Application:

```
s3k_sock_sendrecv(APP_1_CAP_SOCKET, &msg); \
```

In the case above, the application send the message to the monitor and waits for an answer, The monitor receives the msg, performs the comparison and sends the response.

Stack The stack is utilized in this implementation, but somewhat modified. The monitor implements the stack for temporary storage but does the comparison itself. The stack does only provide the monitor with push/pop functionallity.

Memory The monitor does have total ownership of the memory for the running application. This gives the monitor great capabilites including read, write and execute access. Further, this gives the monitor the capabilites neccessary to backup and reset the program from their original state.

parsing Parsing functionality includes parsing the symbol table from the application in the make process before the monitor is compiled. This gives the monitor access to important memory addresses from the application inb order to map out addresses necessary for backup efficiency.

3.1.2 Kernel

For the kernel side implementation we leveraged the fact that a running program has no ability to read or alter the memory allocated by and for the kernel. This is done by allocating the shadow stack as well as memory backups of the apps in the kernel on boot. To access this information, data and do the comparing of the shadow stack we use a multitude of syscalls that all have their own purposes acting as an API to the data that is otherwise unobtainable to the running app. The different syscalls can then be inserted into the `CFI_ENTER` and `CFI_RETURN` macros respectively and that us used to ensure the control flow integrity of the app running.

For each process running there are 2 main structures that are allocated in kernel memory. There is the shadow stack, where information is pushed and popped to according to the earlier explanation, and the backup of the process memory. The shadow stack is used for saving the return address for comparison before return and the backup is a complete memory backup of the process memory and registers that is saved using a syscall. The shadow stack can be accessed using 2 syscalls:

- `s3k_shadow_stack_push(void *ra)` - Which pushes the given return address to the shadow stack in the kernel memory. This function will also trigger the trap-handler if the size of the allocated shadow stack is about to overflow
- `s3k_shadow_stack_pop(void *ra)` - Which pops the top value of the shadow stack, compares the value given with the one on the stack and trips the trap-handler if a mismatch or underflow occurs

Each of the different 3 errors triggers of the trap-handler give their own `mcause` based on the reason for it. This makes it possible for the writer of each app to, by setting a filter in their own trap-handler function, decide what action to take based on the cause of the trigger. The other 2 syscalls are the following:

- `s3k_backup_write()` - Which writes a copy of the current state of the function. Including memory, the stack and the registers. Basically taking a snapshot of the app.

- `s3k_backup_read()` - Which halts the app, writes the backup to the running programs memory and registers and then resumes it. This is basically reverting to the snapshot that was made by the `s3k_backup_write` syscall.

The current default trap handler that is set up using the helper function `setup_cfi_trap` simply runs the `s3k_backup_read` and reverts to the latest backup, if it has been written.

4 Forward Edge

The forward edge protection uses a simple pointer authentication method. Every time a function pointer is declared in the program, it is done so with the macro `PTR_AUTH_INIT(fun)` which takes a function as an argument. The function pointers are stored in custom structs called `function_ptr_X` (where X is a number) that have to be created for the specific function pointer type. When the function pointer is called it is done with the macro `PTR_AUTH(ptr,...)` where *ptr* is an instance of the *function_ptr_X* type and the dots are the arguments to be passed to the function. In addition to creating custom types for each function pointer type, each function pointer type needs to be assigned a *uint64* id. This is done with a generics macro `PTR_TYPE_ID(ptr)` that has to be updated with each new function pointer type that is to be used.

The struct `function_ptr_X` stores a *uint64* signature alongside the function pointer. When `PTR_AUTH_INIT(fun)` is used, the struct is initialized with the function pointer and with a cryptographic signature. The signature is calculated with an XOR operation involving a *uint64* representation of the function pointer address, a *uint64* id derived from `PTR_TYPE_ID(ptr)`, and a secret key. When `PTR_AUTH(ptr,...)` is used the signature is recalculated and compared to the one stored in the struct. If the signatures do not match the function is not called.

5 Design Choices

When implementing the backward edge CFI we decided to use, in both versions of the implementation, a shadow stack functionality. This allowed a separate and, at least for the app, unchangeable space of memory to use for ensuring the control flow of the app.

For the forward edge CFI we decided to use pointer authentication with type discrimination. The cryptographic signature is calculated using XOR encryption due to simplicity. Generics was used in order to derive numeric IDs from the function types.

5.1 Pros

- The shadow stack makes it simple to set up an API for the access to it, making multiple implementations of it easy to create
- The runtime overhead is very small as no cryptographic or other computationally heavy operations are made at runtime
- The memory space used by the shadow stack is variable and can be defined for each app based on the expected amount of recursion
- The macros for enter and return in the functions are not enforced so they can be added or removed at will based on if the writer of the function decides them necessary or not
- The pointer authentication is simple and relatively easy to use.

5.2 Cons

- The size of the shadow stack directly limits the amount of recursion an app can have, as it will otherwise overflow.
- The shadow stack has to be allocated in a memory space we can ensure is not accessible by the running app. Something that either requires an API the attacker could use themselves if the program has unsafe code
- The signature for the function pointer is weak and the key is available to the running app.
- Using additional function types requires extra boilerplate to be written by the programmer.

6 Mitigation of Vulnerabilities

The shadow stack implementation and the macros used in the beginning and end of each function help determine if a change has happened to data that should never change during the execution of a function. This data being the return address of the function. When a function is called the instruction address to where the app should continue to work after the execution is added to a register in risc-V. So that when the function is done and returns it simply puts the program counter to that address and the app continues on its path. This address should never change during the execution and the shadow stack makes it possible for us to verify that it has not changed before letting the function return. If an adversary would use a buffer overflow for example to overwrite this return address, this will be detected before letting the function return to the, possibly malicious, return address. Instead an action, of the app creators choosing, can be executed.

6.1 Limitations

Even though a shadow stack combined with pointer authentication can be an effective combination for control flow integrity, the combination by itself is not an exhaustive solution for vulnerable programs and exploits are still possible.

This combination only protects against exploits attempting to alter control flow. Attacks including, race conditions, logic flaws, and data-oriented attacks do not need to alter control flow and are therefore not prevented in any way by our implementation. Furthermore the forward-edge protection provided by our pointer authentication implementation still allows certain cases of forward-edge control flow altering. Type inclusion in the authentication prevents overwriting a function pointer with another authenticated function pointer of another type. However, if the new function pointer is authenticated and is of the same type as the one being overwritten the authentication will succeed and thus, the control flow altered. Additionally, control-flow altering that utilize jump statements instead of return statements or function pointers are not prevented by our implementation.

7 Individual Contributions

7.1 Jack Lindberg

My contribution to the project was creating the implementation of the cfi control in the kernel. I created an API that made it possible for apps to save and read only the necessary information, to their function, from the kernels own memory. As well as ensured that they had all of these available through syscalls. I also implemented 2 functionalities that were not available to the kernel before. Those where restarting a function from a set snapshot and triggering the trap handler at will. The trap handler trigger can also use a cause that can be defined outside of the regular risc-V error codes. The backup/snapshot write and read are available through syscalls to the app but the trap-handler trigger is only a functionality the kernel has and that is, in this project, used by the cfi comparison

function as an action upon a return address mismatch or shadow stack error. I also implemented a variable memory space in the kernels linker.ld file that allocates memory specifically for storing the backups of the running programs. This is however dependent on the size of each process memory allocation, the amount of processes as well as the PID of each process being started at 0 and only increased by one for each new process. In other words there cannot be a process 0 and a process 2. All of these have to be set in the kernel and cannot be set in each project as it is a limitation of the .ld file format and the fact that it has to be configured in kernel and not per project.

7.2 Filip Redhammar

I created the IPC communication layer including how to implement the socket with the macro. I worked with memory capabilities in order to be able to backup the program using the monitor. This included creating a backup memory space. Requirements needed for the monitorv in order to backup a program includes memory read and write in both regions as well as pmp capabilites. I implemented integration for Carls stack as well as parsing of application symbol table before monitor compilation.

7.3 Carl Tydén

I worked on demonstrating the existing vulnerabilities by creating a program including functions demonstrating forward and backward-edge control flow altering by buffer overflow. Additionally I wrote the implementation of a stack data structure used in the shadow stack. Finally i worked on the forward-edge protection consisting of a pointer authentication implementation with type based discriminators.